

UNIVERZITA PARDUBICE

FAKULTA ELEKTROTECHNIKY A
INFORMATIKY

BAKALÁŘSKÁ PRÁCE

2025

Egor Razboinikov

Univerzita Pardubice
Fakulta Elektrotechniky a Informatiky

Digitalizace evidence pracovní doby
Bakalářská práce

ZADÁNÍ BAKALÁŘSKÉ PRÁCE

(projektu, uměleckého díla, uměleckého výkonu)

Jméno a příjmení:	Egor Razboinikov
Osobní číslo:	I21219
Studijní program:	B0688A140009 Informační technologie
Téma práce:	Digitalizace evidence pracovní doby
Zadávací katedra:	Katedra informačních technologií

Zásady pro vypracování

Dokument evidence pracovní doby vyučujícího vyžaduje provázanost rozvrhu s úvazkem vyučujícího při respektování času přestávek. Cílem bakalářské práce je navrhnout, vytvořit, odzkoušet a předat funkční multiplatformní aplikaci jejíž vstupy jsou: rozvrh vyučujícího, čas na oběd, plán dovolených, státní svátky, školní volna a další. Požadovaným výstupem je PDF dokument (tabulka) se všemi náležitostmi potřebnými pro odevzdání. Grafické rozhraní poskytne pohodlné zadávání, umožní dodatečné ruční modifikace a zobrazování náhledu před vygenerováním PDF dokumentu.

Rozsah pracovní zprávy: **min. 30 stran**
Rozsah grafických prací:
Forma zpracování bakalářské práce: **tištěná**

Seznam doporučené literatury:

PŠENČÍKOVÁ, J. Algoritmizace. Kralice na Hané, Computer Media, 2009.
MAREŠ, Martin a VALLA, Tomáš. Průvodce labyrintem algoritmů 2. vydání. Praha: CZ.NIC, 2022. ISBN 978-80-88168-63-8
PECINOVSKÝ, Rudolf. OOP: naučte se myslet a programovat objektově. Brno: Computer Press, 2010. ISBN 978-80-251-2126-9.

Vedoucí bakalářské práce: **Ing. Radek Matoušek**
Katedra informačních technologií

Datum zadání bakalářské práce: **15. prosince 2024**
Termín odevzdání bakalářské práce: **16. května 2025**

L.S.

prof. Ing. Petr Doležel, Ph.D. v.r.
děkan

Ing. Jan Panuš, Ph.D. v.r.
vedoucí katedry

V Pardubicích dne 28. února 2025

Prohlašuji:

Práci s názvem Digitalizace evidence pracovní doby jsem vypracoval samostatně. Veškeré literární prameny a informace, které jsem v práci využil, jsou uvedeny v seznamu použité literatury.

Byl jsem seznámen s tím, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., o právu autorském, o právech souvisejících s právem autorským a o změně některých zákonů (autorský zákon), ve znění pozdějších předpisů, zejména se skutečností, že Univerzita Pardubice má právo na uzavření licenční smlouvy o užití této práce jako školního díla podle § 60 odst. 1 autorského zákona, a s tím, že pokud dojde k užití této práce mnou nebo bude poskytnuta licence o užití jinému subjektu, je Univerzita Pardubice oprávněna ode mne požadovat přiměřený příspěvek na úhradu nákladů, které na vytvoření díla vynaložila, a to podle okolností až do jejich skutečné výše.

Beru na vědomí, že v souladu s § 47b zákona č. 111/1998 Sb., o vysokých školách a o změně a doplnění dalších zákonů (zákon o vysokých školách), ve znění pozdějších předpisů, a směrnicí Univerzity Pardubice č. 7/2019 Pravidla pro odevzdávání, zveřejňování a formální úpravu závěrečných prací, ve znění pozdějších dodatků, bude práce zveřejněna prostřednictvím Digitální knihovny Univerzity Pardubice.

V Pardubicích dne 9. 5. 2025

Egor Razboinikov v.r.

PODĚKOVÁNÍ

Tady chtěl bych poděkovat svému vedoucímu Ing. Radek Matoušek za hezké vedení bakalářské práce a rychle odpovědi. Také rád bych poděkoval svým rodičům a kamarádům za jejich podporu po celou dobu studia.

ANOTACE

Cílem této bakalářské práce je návrh, implementace a ověření funkčnosti multiplatformní aplikace pro digitalizaci evidence pracovní doby vyučujících. Aplikace umožňuje automatizované zpracování dat, jako jsou rozvrh hodin vyučujícího, stanovené přestávky na oběd, plánované dovolené, státní svátky a školní volna. Výsledným výstupem je detailní PDF dokument, který splňuje formální požadavky fakulty a lze jej snadno upravovat prostřednictvím intuitivního grafického rozhraní. V práci je popsána použitá metodika, výběr vhodných technologií a shrnuty výsledky praktického ověření aplikace.

KLÍČOVÁ SLOVA

C#, .NET, AVALONIA UI, MVVM, AXAML, desktopová aplikace, rozvrh hodin, evidence pracovní doby, generování PDF, automatizace.

TITLE

Digitalization of Working Time Records

ANNOTATION

The aim of this bachelor thesis is to design, implement, and verify a functional cross-platform application for digitizing teachers' working time records. The application automates data processing, including teachers' timetables, designated lunch breaks, vacation schedules, public holidays, and school breaks. The output is a detailed PDF document that meets the faculty's formal submission requirements and can be easily modified via an intuitive graphical interface. The thesis describes the applied methodology, technology selection, and summarizes results of the practical evaluation of the application.

KEYWORDS

C#, .NET, WPF, MVVM, AVALONIA, desktop application, timetable, working time tracking, PDF generation, automation.

OBSAH

SEZNAM ILUSTRACÍ	11
SEZNAM ZKRATEK A ZNAČEK	12
TERMINOLOGIE	13
ÚVOD	14
1 TEORETICKÁ ČÁST	15
1.1 Pravidla evidence pracovní doby	15
1.1.1 Pravidla pro měsíční PDF výkaz	15
1.2 Formální model pracovní doby a rozvrhu	16
1.3 Algoritmické jádro	17
1.4 Architektura a vzory	18
1.5 Programovací jazyky	19
1.5.1 Jazyk C#	20
1.5.2 Jazyk SQL	20
1.6 Framework Avalonia UI	21
1.7 Databázové technologie	22
1.7.1 SQLite	22
1.8 Vývojové prostředí	22
1.8.1 Microsoft Visual Studio	22
1.8.2 SQLiteStudio	23
1.9 Architektura desktopové aplikace	23
1.10 Knihovny	24
2 PRAKTICKÁ ČÁST	25
2.1 Programovací jazyky	25
2.1.1 Jazyk C#	25
2.1.2 Jazyk XAML	26
2.2 Framework Avalonia UI	26

2.3	Architektura aplikace a vzor MVVM	27
2.4	Souborová struktura aplikace.....	29
2.4.1	Složka Behaviors	30
2.4.2	Složka Controls.....	31
2.4.3	Složka Converters	31
2.4.4	Složka Data	31
2.4.5	Složka Helpers	32
2.4.6	Složka Messages	32
2.4.7	Složka Models.....	33
2.4.8	Složka Services	33
2.4.9	Složka ViewModels	33
2.4.10	Složka Views	33
2.4.11	Složka Kořenové soubory projektu.....	33
2.5	Struktura databáze.....	33
2.5.1	Tabulka Events	34
2.5.2	Tabulka GlobalSettings.....	34
2.5.2	Tabulka UserSettings	35
2.6	Uživatelské rozhraní aplikace	35
2.6.1	Hlavní stránka aplikace.....	35
2.6.2	Dialogové okno pro vytvoření události	36
2.6.3	Dialogové okno pro změnu události	37
2.6.4	Vizuální prezentace pro den	38
2.6.5	Vizuální prezentace pro týden	39
2.6.6	Vizuální prezentace pro měsíc	40
2.6.7	Globální nastavení	40
2.6.8	Ukázka PDF	41
2.7	Popsání algoritmů	42

2.7.1 Popis algoritmu BalanceDays	42
2.7.2 Popis algoritmu ze služby pro výpočet pracovní doby	45
ZÁVĚR	46
POUŽITÁ LITERATURA	47

SEZNAM ILUSTRACÍ

Nebyla nalezena položka obsahu.

Obrázek 1: Architektura MVVM.....	27
Obrázek 2: Souborová struktura aplikace.....	30
Obrázek 3 Ukázka kódu třídy AppDbContext.....	32
Obrázek 4 Hlavní stránka aplikace	36
Obrázek 5 Dialogové okno pro vytvoření.....	37
Obrázek 6 Dialogové okno pro změnu	38
Obrázek 7 Vizuální prezentace pro týden.....	39
Obrázek 8 Vizuální prezentace pro měsíc	40
Obrázek 9 Globální nastavení.....	41
Obrázek 10 Ukázka PDF	41
Obrázek 11 : Blokový diagram algoritmu	44

SEZNAM ZKRATEK A ZNAČEK

CSV – Comma-Separated Values

PDF –

FPD – Fond Pracovní Doby

DB – Databáze

DI – Dependency Injection

EPD – Evidence Pracovní Doby

EF Core – Entity Framework Core

FK – Foreign Key

GUI – Graphical User Interface

UI – User Interface

JIT – Just-In-Time

MVVM – Model-View-View Model

ORM – Object-Relational Mapping

PDF – Portable Document Format

UI – User Interface

CLR – Common Language Runtime

XAML – Extensible Application Markup Language

RI – Run-Time Information

CI/CD – Continuous Integration / Continuous Deployment

TERMINOLOGIE

Avalonia UI: Multiplatformní framework pro vývoj uživatelského rozhraní aplikací, který umožňuje tvorbu vizuálně konzistentních aplikací běžících na různých operačních systémech, jako jsou Windows, macOS a Linux, za pomoci XAML a programovacího jazyka C#.

Framework: Sada nástrojů, knihoven a standardizovaných postupů, která usnadňuje proces vývoje softwaru poskytováním struktury a základních funkcionalit, umožňujících rychlou implementaci úloh bez nutnosti opakování vytváření stejného kódu.

Relační databáze: Typ databáze ukládající informace v tabulkách s jasně definovanými vztahy (např. vazby jedna k jedné, jedna k mnoha), umožňující efektivní ukládání, vyhledávání a správu dat.

Programovací paradigma (MVVM): Model-View-ViewModel představuje návrhový vzor programování, který jasně odděluje uživatelské rozhraní (View), datovou logiku (Model) a komunikační logiku (ViewModel), což vede k lepší udržitelnosti a škálovatelnosti aplikace.

Uživatelské rozhraní (UI): Vizuální a interaktivní část aplikace, navržená pomocí frameworku Avalonia UI, která umožňuje uživatelům jednoduše komunikovat s programem prostřednictvím grafických prvků, jako jsou tlačítka, formuláře, menu a další ovládací prvky.

Integrované vývojové prostředí (IDE): Softwarový nástroj kombinující editor kódu, kompilátor, debugger a nástroje pro správu verzí, používaný při vývoji aplikací.

Systémy pro správu verzí (VCS): Nástroje umožňující sledování změn kódu, uchování historie verzí, spolupráci mezi vývojáři a obnovu předchozích verzí kódu v případě potřeby.

ÚVOD

Evidence pracovní doby na vysokých školách musí respektovat přesně stanovená pravidla: provázanost rozvrhu s pracovním úvazkem, povinné přestávky, státní svátky, školní volna a různé formy nepřítomnosti. Měsíční výkaz proto musí zachovávat správný fond pracovní doby. Vzhledem k proměnlivosti rozvrhů a neúplným týdnům je žádoucí systém, který tato omezení konzistentně a předvídatelně uplatňuje.

Tato bakalářská práce se zabývá návrhem a implementací multiplatformní desktopové aplikace „Digitalizace evidence pracovní doby“. Aplikace přebírá rozvrh vyučujícího z informačního systému STAG ve formátu CSV a doplňuje jej o související údaje, například přestávky, státní svátky, školní volna či plánované nepřítomnosti. Jádrem řešení jsou algoritmy, které automaticky rozvrhují a vyvažují pracovní dobu tak, aby byly splněny požadavky fakulty, a v rámci stanovených pravidel případně upravují čas příchodu a odchodu nebo délku polední přestávky. Výstupem je standardizovaná tabulka pracovní doby ve formátu PDF.

Teoretická část práce vymezuje pravidla evidence pracovní doby a popisuje použité modely a klíčové algoritmy; dále podává přehled použitých technologií a jejich role v systému. Praktická část představuje architekturu a implementaci aplikace, popisuje uživatelské rozhraní a scénáře práce s pracovní dobou a odůvodňuje volbu technologií s ohledem na požadavky na funkčnost, údržbu a multiplatformní dostupnost. Zahrnuje rovněž implementaci importu ze systému STAG, implementaci generátoru PDF a ověření funkčnosti na reálných datech vyučujících, včetně vyhodnocení dosažených výsledků.

1 TEORETICKÁ ČÁST

Tato kapitola vymezuje doménu evidence pracovní doby ve vysokoškolském prostředí, formuluje pravidla, jimž musí výsledný výkaz odpovídat, a popisuje formální model dat a zásady algoritmického zpracování. Zároveň představuje související architektonické vzory a technologický rámec na obecné úrovni a stanovuje kritéria správnosti i metriky, podle nichž lze posuzovat kvalitu výstupů.

1.1 Pravidla evidence pracovní doby

Evidence pracovní doby převádí časově orientované skutečnosti – zejména výuku, započítatelnou práci, přestávky a nepřítomnosti – do standardizovaného měsíčního výkazu. Základními pojmy jsou pracovní doba, kterou chápeme jako interval v rámci pracovního dne započítávaný do fondu pracovní doby, a fond pracovní doby (FPD) představující měsíční či týdenní normu odvozenou z kalendáře a úvazku pracovníka. Povinné přestávky jsou minimální intervaly odpočinku, které se do pracovní doby nezapočítávají a musí být umístěny mimo vyučovací bloky. Státní svátky a školní prázdniny se evidují ve zvláštním režimu a promítají se do součtů podle interních pravidel. Mezi nepřítomnosti řadíme zejména dovolenou (D), nemoc (N), ošetřování (OČR), návštěvu lékaře (L), svátky a ostatní dny pracovního klidu (S) a pracovní cestu (PC). Termínem kolize označujeme nežádoucí překryv dvou neslučitelných intervalů, typicky překryv výuky s nepřítomností či souběh dvou záznamů pracovní doby. Cílem je vytvořit dokument, který je v souladu s rozvrhem, respektuje normy pracovní doby a žádné kolize neobsahuje.

1.1.1 Pravidla pro měsíční PDF výkaz

Výkaz musí být jednoznačně provázán s rozvrhem. Začátky a konce pracovní doby i přestávky se uvádějí tak, aby odpovídaly rozvrhu; přestávka se nikdy nečerpá v době výuky a v případě bezprostředně navazujících vyučovacích bloků se čerpá na jejich rozhraní. Zároveň se uplatňuje týdenní norma: u plného úvazku činí součet započítatelné pracovní doby ve týdnu 40 hodin; je-li ve týdnu jeden státní svátek, považuje se týdenní norma za splněnou při 32 hodinách. Je-li pracovní týden rozdělen do dvou měsíců, vykazuje se v každé části poměrná hodnota daná počtem pracovních dnů násobeným 8 hodinami; u zkrácených úvazků se norma upravuje podle pracovní smlouvy.

Na měsíční úrovni musí vždy platit rovnost $\text{Odpracováno} + \text{Přesčas} + \text{Neodpracováno} = \text{FPD}$. Tato rovnost představuje závazné kontrolní pravidlo (invariant) výkazu. Dokument současně nesmí obsahovat překryvy: záznamy se navzájem nepřekrývají a zejména nesmí vzniknout

souběh výuky s nepřítomností ani souběh dvou stavů pracovní doby v témž čase. V případě, že se v jednom časovém intervalu sejde vyučovací blok a záznam nepřítomnosti, má přednost nepřítomnost; vyučovací blok je v daném intervalu pro účely evidence ignorován a nezapočítává se do odpracované doby. Pojem „přesčas“ je vymezen vůči týdenní normě – jde o práci nad 40 hodin týdně a nejde o „napracování“ volna; hodiny nad rámec se ve výsledných součtech promítají tak, aby byla norma týdne zachována a bylo zřejmé, jaká část připadá na odpracováno a jaká na přesčas.

Neodpracovaný čas tvoří souhrn kódovaných nepřítomností, například D, N, OČR, L či PC. Státní svátky se evidují zvláštním režimem a odpovídajícím způsobem vstupují do týdenních i měsíčních součtů. Práce vykonaná o víkendech v rámci pracovní cesty se do výkazu nezapisuje; související náhrady se řeší samostatným cestovním příkazem. Výkaz musí být přehledný a auditovatelný: záznamy jsou strukturovány po dnech, souhrny jsou dostupné na týdenní i měsíční úrovni a lze je jednoznačně zkontrolovat proti fondu pracovní doby.

1.2 Formální model pracovní doby a rozvrhu

Formální model vychází z časové osy s jemným, předem daným krokem (typicky po minutách). Každý záznam je interval vymezený začátkem a koncem, s uvedením typu (např. výuka, započítatelná práce, přestávka, nepřítomnost), původu údajů (rozvrh, systémové pravidlo, uživatelské nastavení) a priority pro řešení kolizí. Vazba na den, týden a měsíc umožňuje kontrolní součty a agregace.

Nad touto reprezentací používáme základní operace: průnik (test kolize), sjednocení (sloučení navazujících intervalů téhož typu), doplněk v rámci pracovního dne (konstrukce přestávek či volných oken) a agregaci (převod denních intervalů na týdenní a měsíční souhrny pro finální výstup).

Model uplatňuje precedenční pravidla: rozvrh výuky má vyšší prioritu než běžná práce; nepřítomnost má vyšší prioritu než obojí. Přestávky se vkládají s ohledem na rozvrh i pracovněprávní limity a při kolizi se posouvají mimo ruční záznamy. Pokud se vyučovací blok potká s nepřítomností, platí nepřítomnost a v daném úseku se práce nezapočítává. Při celodenní zvláštní události (např. dovolená ≥ 8 hodin) se denní pracovní doba pro daný den nesestavuje. O víkendech a ve státní svátky se denní pracovní doba rovněž nedopočítává.

Vstupy a zpracování: Externím vstupem je pouze export rozvrhu. Z něj se převezmou vyučovací bloky jako součást denní pracovní doby. Dále se využívají interní informace o státních svátcích

a globální nebo uživatelská nastavení (příchod, odchod, oběd). Školní prázdniny ani nepřítomnosti se z externích zdrojů nenačítají; projeví se jen tehdy, jsou-li zadány ručně.

Normalizace probíhá tak, že záznamy z rozvrhu se nejprve oříznou na hranice dne. Případné kolize mezi záznamy se pouze označí — záznamy se neslévají ani neupravují. Oběd se určí z nastavení a posune mimo kolize. V časovém rámci mezi příchodem a odchodem se z nepracovních úseků (výuka, přestávky, zvláštní události) odvodí mezery, které tvoří dopočtené úseky denní pracovní doby. Podle potřeby se následně omezí přesčasy úpravou těchto dopočtených úseků.

Výsledkem je měsíční množina intervalů denní pracovní doby (kombinace rozvrhových bloků a dopočtených úseků mimo přestávky), z níž lze přímo sestavit standardizovanou tabulku pracovní doby (PDF) a provést související kontroly. Úplnost kontrol týkajících se prázdnin a nepřítomností odpovídá dostupným vstupům (rozvrh + interní svátky + ruční zadání).

1.3 Algoritmické jádro

Algoritmus vychází z datového modelu intervalů popsaného v kap. 1.2 a z normativních pravidel shrnutých v kap. 1.1. Tato pravidla zde dále nerozvádíme; používáme je jako omezení, která musí být v každém kroku splněna. Cílem je ze vstupů sestavit za daný měsíc konzistentní výkaz tak, aby byly zachovány týdenní normy i měsíční fond pracovní doby a zároveň nevznikaly kolize.

Zpracování začíná převodem vstupů do jednotné interní podoby a jejich normalizací. V této fázi se uplatňují priority stavů: nepřítomnost má vždy přednost před výukou i započitatelnou prací; přestávky se vkládají s ohledem na rozvrh a pracovněprávní limity. Výsledkem je konzistentní množina denních intervalů, u níž lze jednoznačně rozhodovat o překryvech a součtech.

Následuje detekce kolizí, jejímž cílem je ověřit, že se žádné dva neslučitelné záznamy nepřekrývají. V praxi používáme průchod časovou osou s tříděním hran nebo intervalové datové struktury, aby bylo možné o překryvu rozhodovat efektivně i u větších objemů dat. Pokud se kolize objeví, jde o chybný stav; algoritmus pokračuje až poté, co je konflikt vyřešen v souladu s prioritami stavů.

Klíčovým krokem je vyvažování. Pro každý týden se porovná součet započitatelné práce s požadovanou normou. Odchytky se dorovávají minimálními zásahy: posuny hran započitatelných intervalů v povolených mezích nebo redistribucí započitatelné práce mezi dny

téhož týdne. Týdny na rozhraní měsíců se zpracovávají podle svých poměrných částí. Následuje kontrola měsíční kontrolní rovnice; pokud nesedí, provedou se drobné úpravy v rámci zbývajících prostoru, vždy s respektem k prioritám a bez zavedení nových překryvů.

Každá změna je okamžitě validována. Systém kontroluje bezkoliznost, soulad s rozvrhem a přestávkami i správnost součtů. Úpravy, které vedou k porušení pravidel, se zamítají a algoritmus hledá méně invazivní alternativu. Tím se garantuje konzervativní chování: stejné vstupy vedou ke stejným výstupům a počet zásahů zůstává na nezbytném minimu.

Po stabilizaci stavu následuje agregace a export. Denní intervaly se deterministicky převádějí na týdenní a měsíční souhrny a formátují podle šablony PDF. Export nepřidává nové informace; pouze převádí interní reprezentaci do standardizované, administrativně použitelné podoby.

Z hlediska výpočetní náročnosti dominuje detekce překryvů a slučování intervalů, typicky v kvazilineárním čase vzhledem k počtu záznamů. Lokální posuny v rámci dne či týdne jsou lineární v počtu upravovaných intervalů. Důraz na determinismus a minimalitu zásahů usnadňuje auditovatelnost a dohledatelnost vazby mezi rozvrhem, pravidly a výsledným výkazem.

1.4 Architektura a vzory

Model–View–ViewModel (MVVM) je architektonický vzor určený k oddělení prezentační logiky od uživatelského rozhraní. V kontextu deklarativních UI (XAML/AXAML) je View čistě vizuální vrstva, ViewModel vystavuje stav a chování formou vlastností a příkazů a Model nese doménová data a pravidla. Klíčovým mechanismem je data binding, který zajišťuje synchronizaci mezi View a ViewModelem přes notifikace změn (typicky *INotifyPropertyChanged*) a minimalizuje nutnost kódu „za oknem“ [3], [11]. V prostředí .NET k tomu poskytuje jazyk C# typovou bezpečnost, idiomatickou podporu událostí a asynchronní operace (*async/await*), což je pro UI logiku a práci s I/O (import, generování) zásadní [1].

Princip fungování v MVVM lze stručně popsat takto: View je deklarativně definováno v AXAML, ViewModel drží stav prezentace (např. vybraný den, agregované součty) a vystavuje příkazy (např. „přepočti“, „exportuj“). Změna ve View (uživatelská akce) se promítne do ViewModelu přes vazbu, který provede doménové operace nad Modelem a aktualizovaný stav se opět automaticky projeví ve View. Tento tok dat a událostí je v Avalonia UI standardně podporován; projekt může navíc využít knihovny orientované na MVVM (např. ReactiveUI) pro reaktivní příkazy, pozorovatelné stavy a silnější typovou kontrolu vazeb [3], [10]. Interakce lze dále strukturovat pomocí Behaviors, které zapouzdřují opakovatelná prezentační chování

bez „lepení“ logiky do kódu View; tento přístup je zdokumentován jak v ekosystému WPF, tak v Avalonia UI [11], [9].

Výhody MVVM v desktopových XAML frameworkách zahrnují jasné oddělení zodpovědností (UI vs. logika), dobrou testovatelnost ViewModelu bez závislosti na UI, snadnější refaktorování a možnost paralelní práce týmových rolí (návrh rozhraní × programování logiky). Datové vazby snižují množství „lepidlového“ kódu, usnadňují opětovné použití komponent a přirozeně podporují reaktivní aktualizace náhledů (den/týden/měsíc) po změně stavu [3], [10], [11]. Silnou stránkou je i návaznost na C# a .NET (asynchronní operace, události, generika) [1].

Nevýhody a úskalí plynou z toho, že vazby jsou deklarativní a ViewModely mohou růst do šířky. U jednoduchých obrazovek může být MVVM nadbytečné; u složitých scénářů hrozí „přerůstání“ ViewModelů, obtížnější ladění chyb v bindingzích a skryté vazby, které nejsou zřejmé z kódu. Doporučuje se proto důsledné členění na menší, soudržné ViewModely, diagnostika a logování bindingů a využití osvědčených postupů pro chování (Behaviors) a příkazy, jak je popisují dokumentace XAML frameworků [11], [3], [9].

Stručně k MVC a rozdíly vůči MVVM. V klasickém Model–View–Controller (MVC) zpracovává vstupy uživatele Controller, který řídí tok událostí a aktualizuje Model i View. V MVVM tuto roli přebírá ViewModel v kombinaci s datovými vazbami, které přímo propojují stav View a ViewModelu; explicitní controller nebývá potřeba. V prostředí deklarativního UI (AXAML/XAML) je proto MVVM přirozenější volbou, zatímco MVC je typické spíše pro imperative-UI nebo webové architektury, kde controller a směřování požadavků tvoří jádro interakcí [3], [11].

Protože aplikace využívá Avalonia UI (deklarativní AXAML) a .NET/C#, MVVM poskytuje přirozený rámec pro oddělení prezentační logiky, validací a doménových pravidel od zobrazení, s přímou podporou datových vazeb, příkazů a reaktivních stavů. Doporučení z dokumentace Avalonia UI, ReactiveUI a XAML Behaviors poskytují ucelené postupy pro návrh, testování a udržitelnost takových aplikací [3], [10], [9], [11], [1].

1.5 Programovací jazyky

Programovací jazyky představují základní nástroj pro tvorbu softwaru. Umožňují vývojářům formulovat algoritmy a instrukce ve formě, kterou lze převést do strojového kódu a vykonat na konkrétním hardware. Rozlišují se jazyky nízkoúrovňové – blízké strojovému kódu,

vysokoúrovňové – blízké přirozenému jazyku a specifické evidence pracovní doby – navržené pro úzký okruh úloh.

Volba konkrétního jazyka závisí na požadavcích projektu, cílové platformě, výkonových potřebách a zkušenostech vývojového týmu. Správně zvolený jazyk určuje efektivitu vývoje i udržitelnost softwaru.

1.5.1 Jazyk C#

C# je moderní, vysokoúrovňový a objektivě orientovaný jazyk vyvinutý společností Microsoft. Vznikl na přelomu 20. a 21. století jako součást platformy .NET a rychle se stal jedním z klíčových jazyků pro vývoj desktopových, webových i mobilních aplikací.

Mezi jeho hlavní rysy patří přísné typování, automatická správa paměti pomocí garbage collectoru a bohatá standardní knihovna. C# podporuje více programovacích paradigmat, nejčastěji objektivě orientované, ale také funkcionální konstrukce, například LINQ a asynchronní programování – async, await.

Výhodou jazyka je i jeho multiplatformní charakter. Moderní verze .NET umožňují provoz aplikací nejen na Windows, ale i na Linuxu a macOS, což rozšiřuje možnosti jeho použití. Díky přehledné syntaxi, rozsáhlé dokumentaci a širokému ekosystému nástrojů a frameworků je C# vhodnou volbou jak pro malé projekty, tak pro rozsáhlé podnikové systémy.

1.5.2 Jazyk SQL

Structured Query Language (SQL) je standardizovaný jazyk určený pro práci s relačními databázemi. Jeho počátky sahají do 70. let, kdy byl vyvinut společností IBM jako nástroj pro efektivní správu a dotazování nad velkými objemy dat. Díky deklarativní povaze SQL mohou vývojáři a správci databází formulovat požadavky na data bez nutnosti popisovat přesný postup, jak mají být získána nebo zpracována.

Příkazy SQL se tradičně dělí do několika hlavních kategorií:

- **Data Definition Language**

zahrnuje příkazy CREATE, ALTER a DROP, které slouží k vytváření, úpravám či mazání databázových objektů, jako jsou tabulky, indexy a pohledy.

- **Data Manipulation Language**

tvorí ji příkazy SELECT, INSERT, UPDATE a DELETE, umožňující vyhledávání,

vkládání, změnu a odstraňování dat v tabulkách.

- **Data Control Language**

obsahuje příkazy GRANT a REVOKE pro správu uživatelských oprávnění a přístupových práv k databázovým objektům.

- **Transaction Control Language**

zahrnuje příkazy BEGIN TRANSACTION, COMMIT a ROLLBACK, které zajišťují řízení transakcí, tedy souborů operací, jež se provedou buď celé, nebo se v případě chyby vrátí databáze do původního stavu, čímž se zachovává konzistence dat.

Databázový server při zpracování SQL dotazu provede jeho analýzu, optimalizaci a následné vykonání nad uloženými daty. Tento proces je pro uživatele zcela transparentní a umožňuje efektivní práci i s rozsáhlými datovými soubory.

1.6 Framework Avalonia UI

Frameworky tvoří základní stavební kameny moderních softwarových projektů. Poskytují připravené struktury a knihovny, které urychlují vývoj a podporují konzistentní návrh aplikací.

Pro tuto práci byl zvolen framework Avalonia UI, otevřený multiplatformní nástroj pro tvorbu desktopových aplikací. Jeho architektura vychází z konceptů známých z Windows Presentation Foundation (WPF), ale rozšiřuje je i na Linux a macOS.

Rozhraní aplikace se v Avalonia definuje deklarativně pomocí jazyka XAML, resp. jeho varianty s příponou *.axaml. V těchto souborech se popisuje struktura a vzhled uživatelského rozhraní, zatímco logika je implementována v jazyce C#. Tento princip umožňuje oddělení prezentační a logické vrstvy a podporuje architektonický vzor MVVM.

Avalonia poskytuje širokou paletu ovládacích prvků, mechanismus datových vazeb, podporu stylů a šablon a také možnost vytvářet responzivní rozhraní. Díky vlastní vykreslovací vrstvě je dosaženo konzistentního vzhledu aplikací napříč platformami.

1.7 Databázové technologie

Databázové systémy slouží k ukládání a správě dat ve strukturované podobě. Nejčastěji využívaným přístupem jsou relační databáze, které ukládají data do tabulek propojených relacemi a pro práci s daty používají jazyk SQL.

1.7.1 SQLite

SQLite je lehká vestavěná relační databáze, jejíž první verze vznikla roku 2000. Všechna data ukládá do jediného souboru na disku, nevyžaduje samostatný server ani složité nastavení. Díky tomu je přenosná, rychlá a vhodná pro široké spektrum aplikací – od mobilních zařízení přes desktopové programy až po vestavěné systémy.

SQLite podporuje plnou transakční integritu, umožňuje paralelní čtení a je známá svým malým paměťovým i diskovým nárokem. V praxi se používá pro lokální úložiště dat, konfigurace či mezipaměti a je součástí mnoha frameworků i operačních systémů.

1.8 Vývojové prostředí

Vývojové prostředí je software, který poskytuje vývojářům komplexní sadu nástrojů pro tvorbu, ladění a nasazení aplikací. Typické IDE integruje editor zdrojového kódu, překladač nebo interpret, ladicí nástroje a správu projektů do jednoho uživatelského rozhraní. Cílem je zefektivnit práci programátora, snížit počet chyb a zrychlit celý vývojový proces.

Moderní vývojová prostředí často obsahují funkce pro inteligentní doplňování kódu, refaktoring, správu verzí, integrované testování a profilování výkonu. Podporují více programovacích jazyků a umožňují rozšiřování funkcionality prostřednictvím modulů a doplňků. Výběr konkrétního IDE závisí na používané technologii, požadavcích projektu a preferencích vývojáře.

1.8.1 Microsoft Visual Studio

Microsoft Visual Studio patří mezi přední integrovaná vývojová prostředí (IDE), která umožňují vývojářům pokrýt celý životní cyklus softwaru – od psaní a ladění kódu až po nasazení a správu verzí. Díky podpoře řady programovacích jazyků (C#, C++, Visual Basic, F#, Python a dalších) lze v jednom řešení snadno kombinovat různé technologie. Visual Studio poskytuje funkce jako IntelliSense pro inteligentní dokončování kódu, refaktoring pro rychlou úpravu struktury projektů, integrovaný debugger s možností krokování napříč vlákny a profilování výkonu aplikace.

IDE je navrženo tak, aby plynule spolupracovalo s distribuovanými systémy správy verzí – od Git a GitHub přes Azure DevOps až po Subversion nebo Mercurial. Umožňuje vizuální sledování změn, práci s větvemi a slučování kódu přímo v prostředí, bez nutnosti samostatných nástrojů. Rozšíření a doplňky navíc umožňují připojit nástroje pro Docker, Kubernetes, testování jednotek, analýzu kvality kódu či integraci s cloudovými službami Azure, AWS nebo Google Cloud.

Visual Studio dále zahrnuje návrháře UI pro WPF, WinForms i webové technologie, integrované editory pro SQL, JSON či XML a správce balíčků NuGet pro snadné přidávání třetích stran. Funkce Live Share navíc umožňuje vzdálenou kolaboraci v reálném čase, což vede k efektivnější týmové práci a rychlejšímu řešení problémů.

1.8.2 SQLiteStudio

SQLiteStudio je multiplatformní grafické rozhraní určené ke správě SQLite databází, které nevyžaduje instalaci serveru a pracuje přímo se souborem databáze. Jeho přehledné GUI dovoluje rychle vytvářet, měnit či mazat tabulky, indexy a pohledy, prohlížet i upravovat záznamy a zároveň exportovat výsledky dotazů do formátů CSV, SQL či HTML.

Vestavěný SQL editor podporuje syntaktické zvýraznění, automatické doplňování a historii dotazů, což usnadňuje psaní i ladění složitých SQL příkazů. SQLiteStudio umí zobrazit datový model v podobě ER diagramu, spravovat trigger funkce a definovat cizí klíče. Součástí nástroje je také vizuální návrhář schématu a debugger pro procedurální rozšíření SQLite.

Pro zálohování a obnovu databáze stačí jediný klik, nástroj navíc umožňuje hromadnou úpravu dat, filtrování či seskupování přímo v tabulce. Díky podpoře importu dat z externích zdrojů (CSV, SQL dump) a integrovanému reportingu je SQLiteStudio oblíbenou volbou pro rychlý vývoj prototypů, správu lokálních databází i pro nasazení ve vestavěných systémech.

1.9 Architektura desktopové aplikace

Architektura desktopové aplikace určuje strukturu celého softwarového řešení, definuje způsob komunikace mezi jeho jednotlivými částmi a zajišťuje jeho udržitelnost, rozšiřitelnost a výkon. Dobře navržená architektura odděluje prezentační vrstvu od aplikační logiky a datové vrstvy, čímž zvyšuje čitelnost kódu, usnadňuje testování a umožňuje efektivní týmovou spolupráci.

U desktopových aplikací se nejčastěji využívají následující přístupy:

- **Vrstvená architektura,**
dělí aplikaci na prezentační, logickou a datovou vrstvu.
- **Model–View–ViewModel**
hojně používaný ve frameworku Avalonia či WPF, kde model obsahuje data, View definuje vizuální rozhraní a ViewModel propojuje obě vrstvy.
- **Plugin-based architektura**
umožňující dynamické rozšiřování funkcí pomocí modulů.
- **Event-driven přístup**
stavějící komunikaci na zpracování událostí

1.10 Knihovny

Knihovny představují znovupoužitelné softwarové komponenty, které rozšiřují funkcionalitu aplikací bez nutnosti implementace všech částí kódu od začátku. Mohou obsahovat hotové funkce, třídy či šablony pro řešení specifických úloh.

V prostředí .NET se pro správu knihoven využívá systém NuGet, jenž umožňuje snadnou instalaci, aktualizaci a odstraňování závislostí. Použití knihoven zrychluje vývoj, snižuje chybovost a zajišťuje vyšší kvalitu softwaru, protože vývojáři pracují s prověřenými a otestovanými komponentami.

2 PRAKTICKÁ ČÁST

V této kapitole je podrobně popsána realizace aplikace, která umožňuje evidovat a vyhodnocovat pracovní dobu vyučujících. Zaměřuje se na volbu programovacích jazyků, použitý framework, architekturu aplikace a návrhový vzor MVVM. Dále jsou rozebrány klíčové části implementace včetně práce s databází, importu rozvrhu, generování výstupů a uživatelského rozhraní.

2.1 Programovací jazyky

Volba programovacího jazyka představuje jeden z nejzásadnějších kroků při návrhu softwarového řešení. Ovlivňuje nejen architekturu samotné aplikace, ale také rychlost a efektivitu jejího vývoje, možnosti budoucí údržby a rozšiřitelnosti i přenositelnost mezi jednotlivými platformami. V rámci této práce byla zvolena kombinace dvou jazyků, a to C# a XAML. V prostředí Avalonia UI označovaný příponou .axaml. Tyto jazyky se navzájem doplňují – C# zajišťuje implementaci aplikační logiky a práci s daty, zatímco XAML slouží k deklarativní definici uživatelského rozhraní a jeho propojení s logikou. Společně tvoří ucelený ekosystém, který umožňuje efektivně oddělit prezentační a logickou vrstvu, a přitom zachovat jejich úzkou integraci

2.1.1 Jazyk C#

Jazyk C# byl pro tuto práci zvolen především díky své kompatibilitě s multiplatformním prostředím .NET 8 a přímé podpoře frameworku Avalonia UI. V aplikaci zajišťuje veškerou aplikační logiku a funguje jako páteř celého systému. Pomocí C# jsou implementovány modely reprezentující vyučující, jednotlivé položky rozvrhu i samotnou docházku. Dále zde probíhá zpracování dat získaných z importovaných souborů a jejich ukládání do databázové vrstvy. Klíčovou roli hraje také ve ViewModelech, které propojují uživatelské rozhraní s daty a poskytují uživateli funkce, jako je filtrování rozvrhu, kontrola odpracovaných hodin či vyhodnocování přestávek.

Významné je i využití asynchronního programování, které umožňuje plynulé načítání dat z databáze nebo zpracování souborů bez blokování uživatelského rozhraní. Díky tomu je aplikace responzivní a schopná reagovat na akce uživatele i během složitějších operací. V rámci projektu je jazyk C# využíván rovněž pro generování oficiálních výkazů v podobě PDF dokumentů, a to prostřednictvím knihovny QuestPDF, která poskytuje pokročilé možnosti sazby a formátování.

Výhodou tohoto jazyka je silný typový systém, jenž umožňuje eliminovat velkou část chyb již při kompilaci, a bohatý ekosystém dostupných knihoven. Tyto vlastnosti výrazně usnadnily

implementaci aplikace, protože mnohé funkce bylo možné řešit pomocí existujících nástrojů namísto jejich tvorby od začátku. Celkově lze konstatovat, že volba jazyka C# přispěla k vytvoření spolehlivého, přehledného a snadno rozšiřitelného softwarového řešení.

.

2.1.2 Jazyk XAML

Druhým pilířem celého projektu je deklarativní značkovací jazyk XAML, konkrétně jeho varianta s příponou .axaml, která je součástí frameworku Avalonia UI. Tento jazyk byl navržen především pro popis uživatelského rozhraní a jeho hlavní přínos spočívá v oddělení prezentační vrstvy od aplikační logiky. V rámci projektu je v souborech .axaml popsána struktura jednotlivých oken a prvků rozhraní, jako jsou navigační panely, formuláře nebo kalendářové zobrazení. Logika, která zajišťuje reakce na uživatelské akce, je pak implementována v jazyce C#, což umožňuje čisté oddělení zodpovědností a usnadňuje údržbu celého kódu.

Specifickým rysem XAML je těsná integrace s architektonickým vzorem MVVM. Pomocí mechanismu datových vazeb dochází k automatické synchronizaci mezi vizuálními prvky a datovým modelem, což znamená, že změna v datech se okamžitě projeví v uživatelském rozhraní, aniž by bylo nutné provádět manuální aktualizace. Tento princip byl využit například při tvorbě kalendáře, kde jsou jednotlivé položky rozvrhu dynamicky načítány z databáze a zobrazovány přímo uživateli.

Dalším přínosem XAML je možnost využívat šablony a styly, které zajišťují jednotný vizuální vzhled celé aplikace. To je praktické zejména u rozsáhlejších projektů, kde je potřeba udržet konzistenci uživatelského rozhraní, a přitom umožnit snadné úpravy designu. V této práci byly šablony využity pro definici vzhledu opakujících se prvků, zatímco styly sjednocují grafické zpracování aplikace jako celku.

2.2 Framework Avalonia UI

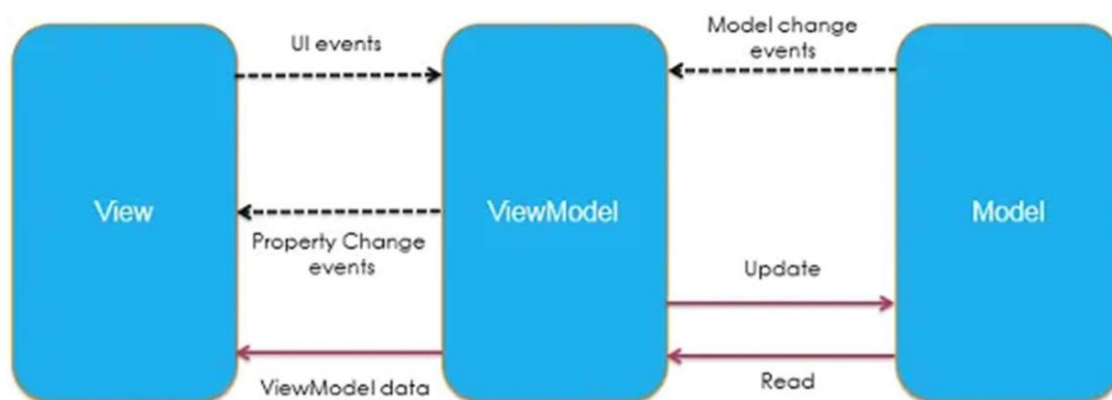
Pro implementaci uživatelského rozhraní byla zvolena knihovna Avalonia UI, která umožňuje vytvářet moderní desktopové aplikace fungující na systémech Windows, Linux i macOS. Hlavním důvodem pro její použití byla právě multiplatformní povaha a také podobnost s technologiemi známými z prostředí WPF, což vývojářům usnadňuje práci. Avalonia poskytuje vlastní vykreslovací vrstvu, díky níž je možné zachovat jednotný vzhled aplikace napříč platformami, aniž by bylo nutné provádět složité úpravy kódu.

V rámci této práce byla Avalonia UI využita k tvorbě všech částí grafického rozhraní. Zajišťuje zobrazení hlavního okna, navigačních prvků i detailního kalendáře, kde je možné přepínat mezi denním, týdenním a měsíčním přehledem. Významnou roli hrají i interaktivní formuláře, které uživateli umožňují nastavovat parametry pracovního dne, zadávat přestávky nebo upravovat rozvrh. Avalonia zároveň poskytuje pokročilé možnosti datových vazeb a příkazů, což zjednodušuje propojení uživatelského rozhraní s logikou implementovanou v jazyce C#.

Neméně důležitou výhodou frameworku je podpora stylů, šablon a responzivního designu, díky nimž lze rychle měnit vizuální podobu aplikace. Tyto vlastnosti přispívají k udržitelnosti projektu a zajišťují, že aplikace působí konzistentně a profesionálně i při budoucím rozšiřování. Celkově lze říct, že Avalonia UI poskytla dostatečně robustní základ pro implementaci přehledného a funkčně bohatého rozhraní, které odpovídá potřebám uživatelů.

2.3 Architektura aplikace a vzor MVVM

Struktura aplikace je postavena na architektonickém vzoru MVVM, který je pro Avalonia UI přirozenou volbou. Tento vzor umožňuje důsledné oddělení prezentační vrstvy od logiky a poskytuje jasný rámec pro organizaci kódu. Díky tomu lze aplikaci snadno rozšiřovat, testovat a udržovat, aniž by došlo k propojení částí, které by měly zůstat nezávislé.



Obrázek 1: Architektura MVVM

Model tvoří perzistentní entity mapované pomocí Entity Framework Core na SQLite databázi. Nejvýznamnějšími třídami jsou Event, UserSettings a GlobalSettings, které představují základní datové struktury aplikace. Třída Event popisuje konkrétní události, UserSettings slouží k uložení individuálních nastavení pro vybraný den a GlobalSettings uchovává výchozí pracovní časy, parametry přestávek, informace o vyučujícím a semestrální režimy. O přístup k

databázi se stará AppDbContext, jenž kromě definice tabulek obsahuje konfiguraci ukládání časových údajů a mapování vztahů mezi entitami.

View představují .axaml soubory, které určují vzhled jednotlivých částí rozhraní. Patří sem například MainWindow.axaml s navigací a kalendářem, DayView.axaml, WeekView.axaml a MonthView.axaml pro různé pohledy na rozvrh, dále dialogy jako CreateEventDialog.axaml nebo GlobalUserSettingsView.axaml. Součástí pohledů jsou i specifická chování (Behaviors), která přidávají interaktivitu – například ShowCollisionMessageBehavior zajišťuje zobrazení upozornění na kolize událostí. Díky těmto chováním zůstává kód čistý a opakovaně použitelný. Uživatelské rozhraní je navrženo tak, aby bylo intuitivní: levý panel hlavního okna nabízí přepínání mezi denním, týdenním a měsíčním pohledem, správu nastavení a generování PDF, zatímco pravý panel zobrazuje samotná data kalendáře.

ViewModel vrstva funguje jako prostředník mezi modelem a pohledem. Hlavní třídou je MainWindowViewModel, který koordinuje načítání dat a přepínání mezi jednotlivými režimy zobrazení. Samostatné ViewModely pro den, týden a měsíc (DayViewModel, WeekViewModel, MonthViewModel) zajišťují logiku a zpracování dat specifické pro danou kategorii kalendáře. Pro práci s globálním nastavením existuje GlobalUserSettingsViewModel, který umožňuje načítat a ukládat pracovní intervaly, parametry přestávek a další údaje. Důležitou roli hraje také CreateEventDialogViewModel, jenž se stará o validaci vstupů při vytváření a editaci událostí, a PdfPreviewViewModel, který připravuje data pro generování a vyvážení pracovních hodin v oficiálním PDF. Veškeré změny dat se do rozhraní propisují pomocí mechanismu data bindingu, což znamená, že aktualizace hodnot ve ViewModelu se okamžitě projeví v odpovídajícím pohledu a uživatel vždy pracuje s aktuálními informacemi.

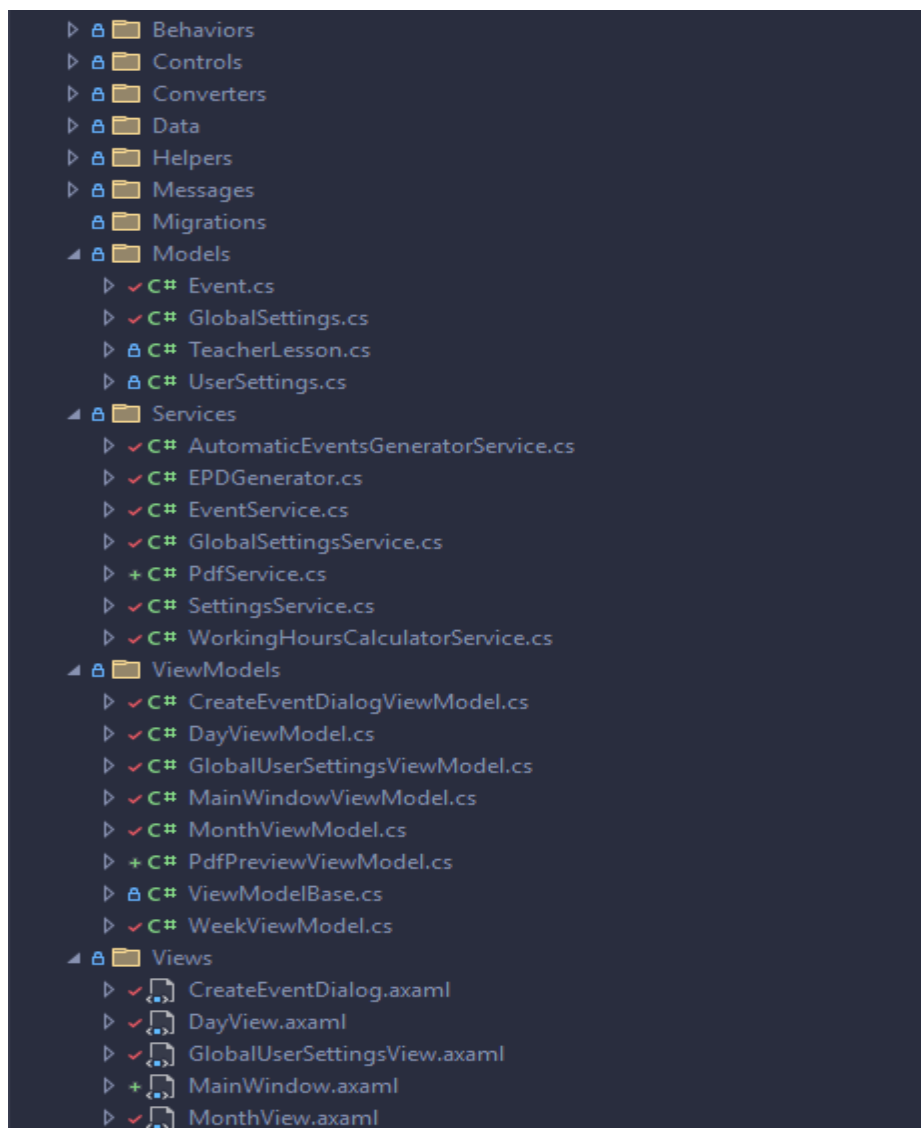
Servisní vrstva doplňuje architekturu o centralizovanou logiku, která by neměla být přímo součástí ViewModelů. Klíčovým prvkem je EventService, který kromě klasických CRUD operací implementuje také algoritmus pro vyvažování odpracovaných hodin mezi jednotlivými dny. Tento mechanismus automaticky vyrovnává přebytky a nedostatky v pracovním fondu tak, aby měsíční bilance odpovídala nastaveným pravidlům, aniž by uživatel musel data upravovat ručně. WorkingHoursCalculatorService zajišťuje výpočty odpracovaného času, AutomaticEventsGeneratorService vytváří výchozí události na základě globálních nastavení a rozvrhu, a PdfService připravuje výsledné výstupy ve formátu PDF. Zvláštní roli hraje komponenta EPDGenerator, která je určena pro načítání rozvrhu vyučujícího ze souboru CSV (exportovaného z IS STAG) a pro převod těchto dat na interní události aplikace. Díky tomu

může uživatel rychle naplnit svůj kalendář skutečným rozvrhem a následně využít ostatní nástroje aplikace pro vyvažování, úpravy a export do oficiálních dokumentů. Dále GlobalSettingsService a SettingsService poskytují rozhraní pro perzistenci uživatelských i globálních konfigurací.

Díky této architektuře je aplikace přehledná, modulární a snadno rozšiřitelná. Každá vrstva má jasně definovanou zodpovědnost: Model obstarává data a jejich uložení, View definuje vzhled a chování uživatelského rozhraní, ViewModel zprostředkovává interakci mezi uživatelem a logikou, a Services centralizují výpočty a komplexní operace. Tento přístup umožnil realizovat robustní desktopovou aplikaci, která je schopná automaticky generovat události, vyrovnávat pracovní fond, zajišťovat multiplatformní běh a poskytovat uživateli komfortní nástroje pro evidenci pracovní doby.

2.4 Souborová struktura aplikace

Struktura projektu je navržena tak, aby jasně oddělovala jednotlivé vrstvy a funkce aplikace a zároveň podporovala zásady přehlednosti, udržovatelnosti a rozšiřitelnosti. Díky tomuto členění lze snadno vyhledat části kódu související s databází, prezentační logikou, uživatelským rozhraním či podpůrnými funkcemi. Projekt je rozdělen do samostatných složek, z nichž každá se zaměřuje na konkrétní oblast.



Obrázek 2: Souborová struktura aplikace

Celkově tato souborová struktura poskytuje jasné oddělení zodpovědností, usnadňuje orientaci v projektu a umožňuje efektivní vývoj i budoucí rozšiřování. Každá složka má svou specifickou úlohu, díky čemuž je aplikace modulární a přehledná, a to jak pro jednoho vývojáře, tak i pro týmovou spolupráci.

2.4.1 Složka Behaviors

Složka Behaviors obsahuje třídy využívající Avalonia.Xaml.Behaviors. Jejich cílem je přidat specifické interakce k uživatelským prvkům, aniž by bylo nutné měnit jejich šablony. Nachází se zde například mechanismy pro zachytávání kliknutí na buňky kalendáře či události, komponenty pro zavírání dialogů a potvrzování jejich akcí nebo chování zajišťující zobrazování chybových a kolizních hlášení. Díky tomuto přístupu zůstává kód rozhraní čistý a interakce lze snadno měnit.

2.4.2 Složka Controls

Další částí je složka Controls, která sdružuje vlastní uživatelské prvky rozšiřující základní komponenty Avalonia UI. Tyto prvky slouží k zobrazení událostí v kalendáři nebo k rozvržení panelů a díky své znovupoužitelnosti přispívají k modulárnosti a konzistenci aplikace.

2.4.3 Složka Converters

Transformace dat mezi ViewModely a zobrazením je soustředěna ve složce Converters. Převodníky implementující `IValueConverter` zde slouží k převodu logických hodnot na viditelnost či barvy, k výpočtu pozic událostí v kalendáři nebo k transformaci časových údajů na grafické rozměry. Tento přístup umožňuje udržet čistý MVVM vzor a logiku transformace oddělit od prezentace.

2.4.4 Složka Data

Datovou vrstvu představuje složka Data obsahující kontext `AppDbContext`. Ten definuje přístup k databázi prostřednictvím Entity Framework Core, deklaruje sady entit pro události a nastavení a zajišťuje připojení k databázovému souboru SQLite uloženému v uživatelském adresáři. Současně mapuje vlastnosti na databázové sloupce a obsahuje konvertory pro jednotné ukládání časových hodnot.

```

public class AppDbContext : DbContext
{
    Ссылка: 14 | Egor Razboinikov, 27 дн. назад | Автор: 1, изменение: 1
    public DbSet<Event> Events { get; set; }
    Ссылка: 5 | Egor Razboinikov, 27 дн. назад | Автор: 1, изменение: 1
    public DbSet<UserSettings> UserSettings { get; set; }
    Ссылка: 5 | Egor Razboinikov, 13 дн. назад | Автор: 1, изменение: 1
    public DbSet<GlobalSettings> GlobalSettings { get; set; }

    Ссылка: 0 | Egor Razboinikov, 27 дн. назад | Автор: 1, изменение: 1
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        string baseDir;
        if (RuntimeInformation.IsOSPlatform(OSPlatform.Windows))
            baseDir = Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData);
        else
            baseDir = Path.Combine(Environment.GetFolderPath(Environment.SpecialFolder.UserProfile), ".config");

        var appDir = Path.Combine(baseDir, "TeacherScheduleApp");
        Directory.CreateDirectory(appDir);

        var dbPath = Path.Combine(appDir, "teacherapp.db");
        optionsBuilder.UseSqlite($"Data Source={dbPath}");
    }

    Ссылка: 0 | Egor Razboinikov, 27 дн. назад | Автор: 1, изменение: 1
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        var dateTimeConverter = new ValueConverter<DateTime, string>(
            v => v.ToString("yyyy-MM-dd HH:mm:ss", CultureInfo.InvariantCulture),
            v => DateTime.ParseExact(v, "yyyy-MM-dd HH:mm:ss", CultureInfo.InvariantCulture)
        );
        modelBuilder.Entity<Event>()
            .Property(e => e.StartTime)
            .HasConversion(dateTimeConverter);

        modelBuilder.Entity<Event>()
            .Property(e => e.EndTime)
            .HasConversion(dateTimeConverter);
        base.OnModelCreating(modelBuilder);
    }
}

```

Obrázek 3 Ukázka kódu třídy AppDbContext

2.4.5 Složka Helpers

Podpůrnou funkci plní složka Helpers, kde jsou umístěny univerzální třídy využívané napříč aplikací. Patří sem například logika pro získání reference na hlavní okno nebo komponenta zpracovávající seznam státních svátků prostřednictvím knihovny Nager.Date.

2.4.6 Složka Messages

Komunikaci mezi částmi systému zajišťuje složka Messages. Obsahuje definice zpráv, které informují ViewModely o důležitých změnách, například o vytvoření nových automatických událostí, dokončení generování PDF či úpravě nastavení. Díky tomuto mechanismu spolu moduly spolupracují bez přímých závislostí.

2.4.7 Složka Models

Datové entity jsou definovány ve složce Models. Nachází se zde třídy popisující události, globální i uživatelská nastavení nebo jednotlivé bloky výuky. Jedná se o jednoduché objekty mapované prostřednictvím Entity Frameworku na databázové tabulky.

2.4.8 Složka Services

Složka Services je jádrem obchodní logiky. Obsahuje služby starající se o správu událostí, generování automatických bloků, výpočet pracovních hodin nebo tvorbu PDF souborů. Zvláštní význam má algoritmus vyvažování pracovní doby, který se snaží rozdělit odpracované hodiny tak, aby se měsíční balance co nejvíce blížila očekávané normě. Do této části patří také třída EPDGenerator, která čte rozvrh učitele ze souboru CSV a převádí jej na databázové události.

2.4.9 Složka ViewModels

Prezentační logiku soustřeďuje složka ViewModels. Každý ViewModel propojuje data s konkrétním uživatelským rozhraním a definuje příkazy pro práci s aplikací. Hlavní ViewModel zajišťuje přepínání mezi denním, týdenním a měsíčním režimem, další spravují dialogy, nastavení nebo náhled PDF.

2.4.10 Složka Views

Vizuální vrstvu tvoří složka Views, v níž se nacházejí šablony .axaml a jejich code-behind. Tyto soubory definují vzhled hlavního okna, jednotlivých kalendářních pohledů a dialogů. Jejich oddělení od logiky usnadňuje úpravy uživatelského rozhraní.

2.4.11 Složka Kořenové soubory projektu

Kořenové soubory projektu obsahují deklarace globálních zdrojů a startovací logiku. App.axaml definuje styly a šablony, jeho code-behind inicializuje hlavní okno a životní cyklus aplikace. Manifest určuje metadata pro Windows, zatímco Program.cs představuje vstupní bod aplikace. ViewLocator pak zajišťuje propojení ViewModelů s odpovídajícími Views. Celkově je struktura navržena tak, aby byla modulární, přehledná a snadno rozšiřitelná.

2.5 Struktura databáze

V dnešní době se bez databáze neobejde žádná aplikace, proto se v této části bakalářské práce zaměříme na její strukturu a návrh. Databáze této aplikace je určena k ukládání různých informací o uživateli. Obsahuje tři tabulky, z nichž každá je zodpovědná za ukládání specifických informací. Tyto tabulky dohromady tvoří páteř databáze a zajišťují správu informací o událostech, nastavení uživatele pro konkrétní den a globální nastavení.

2.5.1 Tabulka Events

Tabulka Events slouží k ukládání všech časových bloků v rozvrhu učitele. Každý řádek má unikátní Id primární klíč a následující sloupce:

- **Title** – popis události, například název předmětu.
- **StartTime, EndTime** – čas začátku a konce události ve formátu yyyy-MM-dd HH:mm:ss.
- **AllDay** – příznak celodenní události (0 nebo 1).
- **Description** – volný popis či poznámka.
- **IsDeleted** – logické smazání (0 = aktivní, 1 = vymazáno).
- **EventType** – číselný kód typu události (práce, oběd, služební cesta apod.).
- **ArrivalTime, DepartureTime, LunchStart, LunchEnd** – detailní časy dne pro případné manuální úpravy.
- **IsAutoGenerated** – příznak, že událost byla automaticky vygenerována (0 nebo 1).
- **HasCollision** – indikátor kolize s jinou událostí (0 nebo 1).
- **AutoGeneratedForDate** – datum, pro které byla událost vygenerována.

2.5.2 Tabulka GlobalSettings

Tabulka GlobalSettings uchovává výchozí nastavení pracovní doby a parametrů pro celý semestr. Každý záznam má unikátní Id a obsahuje:

- **GlobalStartTime, GlobalEndTime** – standardní začátek a konec pracovního dne.
- **EmployeeName, Department** – jméno vyučujícího a název katedry či oddělení.
- **{Day}Arrival, {Day}Departure** – výchozí příchod a odchod. Po čtyři sloupce pro každý pracovní den (pondělí – pátek):
- **{Day}LunchStart, {Day}LunchEnd** – výchozí začátek a konec polední přestávky.
- **MinBreakDuration, MaxBreakDuration** – minimální a maximální délka povolené přestávky.

- **AutoEventNamePreLunch, AutoEventNameLunch, AutoEventNamePostLunch** – přednastavené názvy automaticky generovaných bloků (před obědem, oběd, po obědě).
- **Semester** – identifikátor semestru (např. 0 = zimní, 1 = letní).

2.5.2 Tabulka UserSettings

Tabulka UserSettings ukládá individuální úpravy pracovního dne pro konkrétní datum. Každý záznam má unikátní Id (PK AUTOINCREMENT) a tyto sloupce:

- **Date** – datum ve formátu yyyy-MM-dd.
- **ArrivalTime, DepartureTime, LunchStart, LunchEnd** – ručně zadané časy příchodu, odchodu a začátku/konce oběda pro daný den.

Tyto úpravy mají prioritu nad globálními nastaveními z tabulky GlobalSettings a aplikace je použije při generování a vyvažování událostí.

2.6 Uživatelské rozhraní aplikace

Uživatelské rozhraní aplikace je někdy nejdůležitějším aspektem, který určuje, jak snadná a intuitivní bude pro uživatele interakce s aplikací. Jednoduchý a přímočarý návrh uživatelského rozhraní zajišťuje, že aplikace je intuitivní, snadno se používá. To pomáhá uživateli rychle a snadno pochopit, jak aplikace funguje, a může efektivně využívat její funkce. Význam uživatelského rozhraní by se neměl podceňovat. Koneckonců je to tvář aplikace a měla by být atraktivní. Pokud rozhraní obsahuje velké množství prvků nebo je obtížné se v něm orientovat, může to mít negativní dopad na uživatelský zážitek. Naopak dobře navržené uživatelské rozhraní s jasným vizuálem, a nejen strukturou může pro uživatele hrát velmi velkou roli. Klíčovými prvky dobrého uživatelského rozhraní jsou jednoduchost, přehlednost a konzistence.

2.6.1 Hlavní stránka aplikace

První věc, kterou uživatel uvidí po spuštění aplikace, je její hlavní stránka. Při prvním spuštění aplikace se také automaticky vytvoří události na celý rok. Hlavní stránka je rozdělena na dva hlavní panely. Levý panel slouží k různým interakcím s aplikací. Druhý panel slouží k zobrazení pohledů na daný den, týden nebo měsíc v závislosti na tom, co uživatel vybral v rozevíracím seznamu na levém panelu. Výchozí zobrazení je aktuální den.

Popíšeme podrobně levý panel. V horní části tohoto panelu jsou čtyři tlačítka, která slouží k vyvolání různých funkcí. První tlačítko „Vytvořit událost“ slouží k vyvolání dialogového okna

(jeho popis je uveden níže). Druhé tlačítko „Generovat EPD“ slouží k vyvolání dialogového okna pro načtení souboru .csv s rozvrhem učitele ze STAG. Po výběru souboru se spustí proces vytváření událostí ze souboru. Třetí tlačítko „Globální nastavení“ slouží k otevření okna globálních nastavení (podrobný popis níže). Čtvrté tlačítko „Příklad PDF“ slouží k otevření dialogového okna pro prohlížení dokumentu ve formátu PDF. Za tlačítky se nachází kalendář s možností výběru dne, měsíce a roku. Pokud v tomto kalendáři kliknete na jakýkoli den, aplikace přejde a zobrazí údaje pro tento den. Za kalendářem se nachází rozevírací seznam, který slouží k výběru možnosti zobrazení dne, týdne nebo měsíce. Za seznamem následuje blok nastavení pro konkrétní den. Tam se nacházejí 4 hlavní pole, která slouží k zadání času příchodu, času odchodu a začátku a konce přestávky, a tlačítko, které uloží zadané informace. Stojí za zmínku, že tento blok je viditelný pouze v denním režimu. Za tímto blokem se zobrazují informace o tom, kolik hodin bylo odpracováno z normy.

The screenshot displays the application's main interface. On the left, there is a sidebar with four yellow buttons: "Vytvořit událost", "Generovat EPD", "Globální nastavení", and "Ukázka PDF". Below these buttons is a calendar for May 2025, with the 4th of May highlighted. Under the calendar is a "Rozvrh" dropdown menu. Further down is a "Nastavení pro den:" section for the date 04.05.2025, containing four time input fields: "Příchod:" (00:00), "Odchod:" (00:00), "Oběd od:" (00:00), and "Oběd do:" (00:00), followed by a yellow "Uložit nastavení" button. At the bottom left, it shows "Pracovní doba (Den): 0,0 / 8". The main area on the right is a daily schedule for "Den: 04 kvě 2025". It features a vertical time axis from 00:00 to 18:00 in 1-hour increments, with a corresponding horizontal axis for the day of the week (P, Ú, S, Č, P, S, N).

Obrázek 4 Hlavní stránka aplikace

2.6.2 Dialogové okno pro vytvoření události

Dialogové okno obsahuje následující prvky. První pole slouží k zadání nebo úpravě názvu události. Toto pole je povinné, při pokusu o vytvoření události bez názvu se zobrazí dialogové okno s chybou. Druhé pole již není povinné a slouží k přidání popisu události. Poté následuje tlačítko, které určuje, zda bude událost trvat celý den. Poté následuje blok pro nastavení data a času události. Za blokem pro nastavení data je prvek, který umožňuje vybrat typ události (pracovní, volno, služební cesta atd.). A posledními prvky jsou tlačítko pro zrušení, které zavře okno, a tlačítko pro uložení, které událost vytvoří a okno zavře.

Nová událost

Název

Popis

☐ Celý den

Začátek:

4 květen 2025 0 00

Konec:

4 květen 2025 0 00

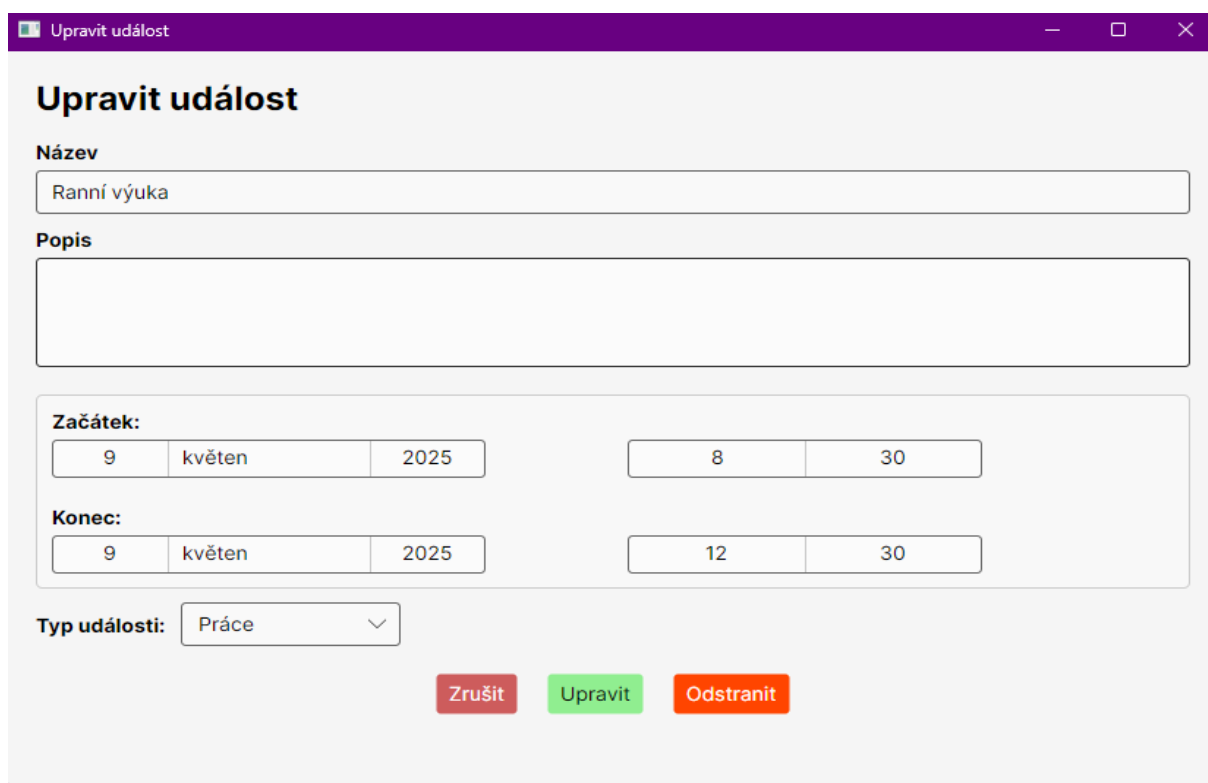
Typ události: Práce

Zrušit Vytvořit

Obrázek 5 Dialogové okno pro vytvoření

2.6.3 Dialogové okno pro změnu události

Dialogové okno pro změnu události obsahuje následující prvky. První pole slouží k zadání nebo změně názvu události. Toto pole je povinné, při pokusu o změnu události bez názvu se zobrazí dialogové okno s chybou. Druhé pole není povinné a slouží k přidání popisu události. Dále následuje blok pro nastavení data a času události. Za blokem pro nastavení data se nachází prvek, který umožňuje vybrat typ události (pracovní, volno, služební cesta atd.). Posledními prvky jsou tlačítka pro zrušení, které zavře okno, tlačítko pro uložení změn, které aktualizuje událost a zavře okno, a tlačítko, které slouží k odstranění události. Před odstraněním události bude uživatel požádán o potvrzení odstranění události.



Upravit událost

Název

Ranní výuka

Popis

Začátek:

9 květen 2025 8 30

Konec:

9 květen 2025 12 30

Typ události: Práce

Zrušit Upravit Odstranit

Obrázek 6 Dialogové okno pro změnu

2.6.4 Vizuální prezentace pro den

Vizuální prezentace pro daný den obsahuje mřížku s hodinami, pracovní interval je barevně odlišen od nepracovního. V horní části jsou tlačítka, která umožňují přepínat mezi dny. Tam se také nachází tlačítko, které přesune uživatele na aktuální den. Stejně tak zobrazuje události pro daný den, pokud nějaké existují.

← Den: 02 kvě 2025 → Dnes	
01:00	
02:00	
03:00	
04:00	
05:00	
06:00	
07:00	
08:00	Ranní výuka
09:00	
10:00	
11:00	
12:00	
	Oběd
13:00	Odpolední výuka
14:00	
15:00	
16:00	
17:00	
18:00	
19:00	

2.6.5 Vizuální prezentace pro týden

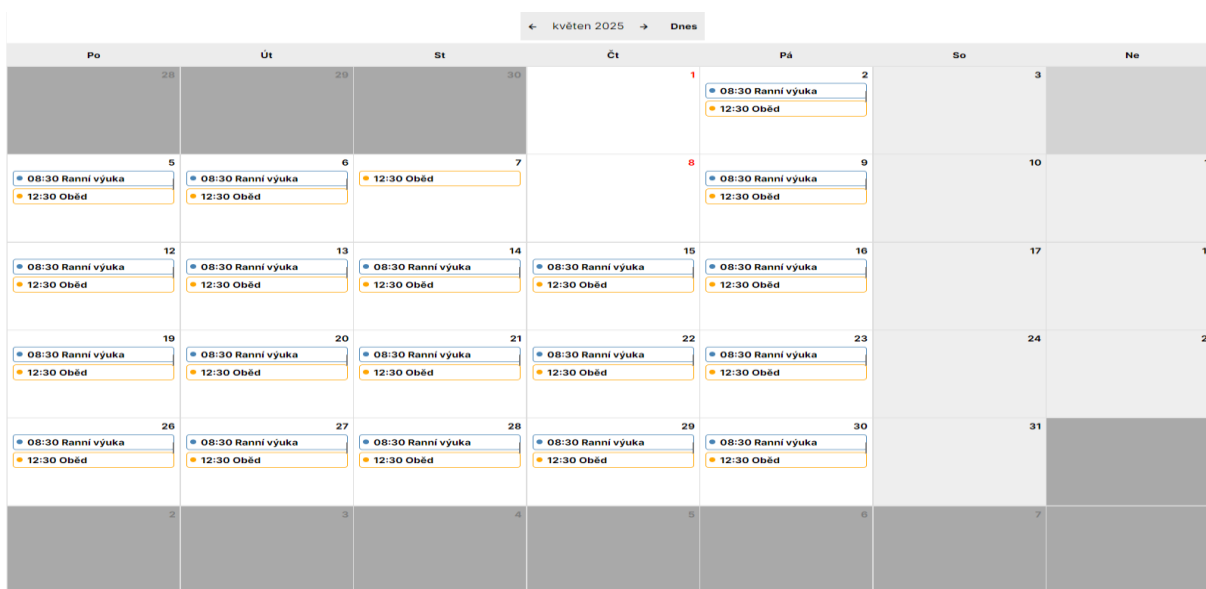
Vizuální prezentace pro týden obsahuje také tabulku s hodinami a dny, které tvoří týden. V horní části jsou tlačítka, která umožňují přepínat mezi týdny. Tam se také nachází tlačítko, které přesune uživatele na aktuální týden. Stejně tak zobrazuje události pro tento týden, pokud nějaké jsou.

← Dnes →							
	Po 28 dub	Út 29 dub	St 30 dub	Čt 01 kvě	Pá 02 kvě	So 03 kvě	Ne 04 kvě
00:00							
01:00							
02:00							
03:00							
04:00							
05:00							
06:00							
07:00							
08:00	Ranní výuka	Ranní výuka	Ranní výuka		Ranní výuka		
09:00							
10:00							

Obrázek 7 Vizuální prezentace pro týden

2.6.6 Vizuální prezentace pro měsíc

Vizuální prezentace pro měsíc obsahuje také mřížku, ale již s dny, které spadají do tohoto měsíce. Státní svátky, víkendy a dny z předchozího nebo následujícího měsíce se liší barvou pozadí nebo barvou čísla dne. V horní části jsou tlačítka, která umožňují přepínat mezi měsíci. Tam se také nachází tlačítko, které přesune uživatele na aktuální měsíc. Stejně tak zobrazuje události pro tento měsíc, pokud nějaké jsou.



Obrázek 8 Vizuální prezentace pro měsíc

2.6.7 Globální nastavení

Globální nastavení obsahuje velké množství polí pro zadání informací o různých časových intervalech a dalších informací o uživateli, které budou použity pro vytváření automatických událostí, generování PDF a tak dále. Podrobný popis začneme tlačítky, která slouží k přepínání mezi zimním a letním semestrem, což uživateli umožňuje zadat různé údaje pro konkrétní semestr. Pole pro zadávání informací jsou stejná pro obě varianty. Dále následuje blok, který odpovídá za globální interval pracovní doby. Poté následuje blok s možností zadání informací o příchodu do práce a odchodu z práce a také času přestávky. Tato pole jsou stejná pro všechny pracovní dny. Záložka další nastavení obsahuje pole pro zadání jména, katedry, které budou použity pro generování v PDF. Pole pro zadání omezení pro přestávku a také jména, která se používají pro vytváření automatických událostí. Na konci se také nachází tlačítko pro uložení.

Globální nastavení uživatele
 Zimní semestr
☒ Zimní ☐ Letní

Globální pracovní doba

Od: 08:00
do: 16:30

Zimní semestr

Pondělí
Úterý
Středa
Čtvrtek
Pátek
Další nastavení

Příchod: 08:00
 Odchod: 16:30
 Oběd, začátek: 12:00
 Oběd, konec: 12:30

Uložit nastavení

Obrázek 9 Globální nastavení

2.6.8 Ukázka PDF

Náhled PDF obsahuje dva prvky, se kterými lze interagovat. První slouží k přepínání mezi měsíci, druhý k uložení souboru do počítače. Také obsahuje obrázek, jak bude stažený soubor vypadat.

2025-05 Save PDF

Evidence pracovní doby, včetně přestávek v práci a práce přesčas

Fond prac. doby:
160 hodin

jméno:
Radek Matoušek

útvar:
Katedra
informačních
technologií

pracovní doba: 08:30–17:00 hod.
 docházka za měsíc: květen 2025

Den	Začátek pracovní doby	1. přestávka		2. přestávka		Konec pracovní doby	Odpracováno	Přesčas	Neodprac.	Poznámka *)
		Od	Do	Od	Do					
1.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	S
2.										
3.										
4.										
5.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
6.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
7.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
8.										
9.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	S
10.										
11.										
12.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
13.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
14.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
15.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
16.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
17.										
18.										
19.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
20.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
21.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
22.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
23.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
24.										
25.										
26.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
27.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
28.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
29.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
30.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
31.										
Celkem							160:00:00	00:00:00	00:00:00	

kontr. č. Z odprac. - přesčas + neodprac. hodin (odpovídá FPD): 160:00:00
 podpis zaměstnance: _____ podpis nadřízeného pracovníka: _____

*) do poznámky: D – dovolená, N – nemoc, OČR – ošetřování, L – lékař, PC – pracovní cesta, S – svátek a ostatní dny pracovního klidu

Obrázek 10 Ukázka PDF

2.7 Popsání algoritmů

V této kapitole podrobně rozebereme klíčové algoritmy použitými v aplikaci. Nejprve uvedeme jejich základní účel a postup, poté krok za krokem popíšeme jejich logiku včetně vstupů, výstupů a případných okrajových podmínek.

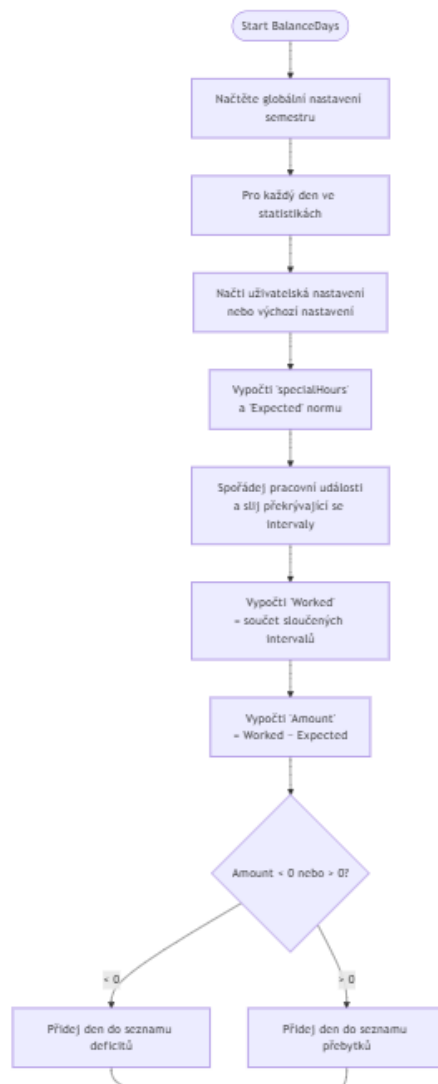
2.7.1 Popis algoritmu BalanceDays

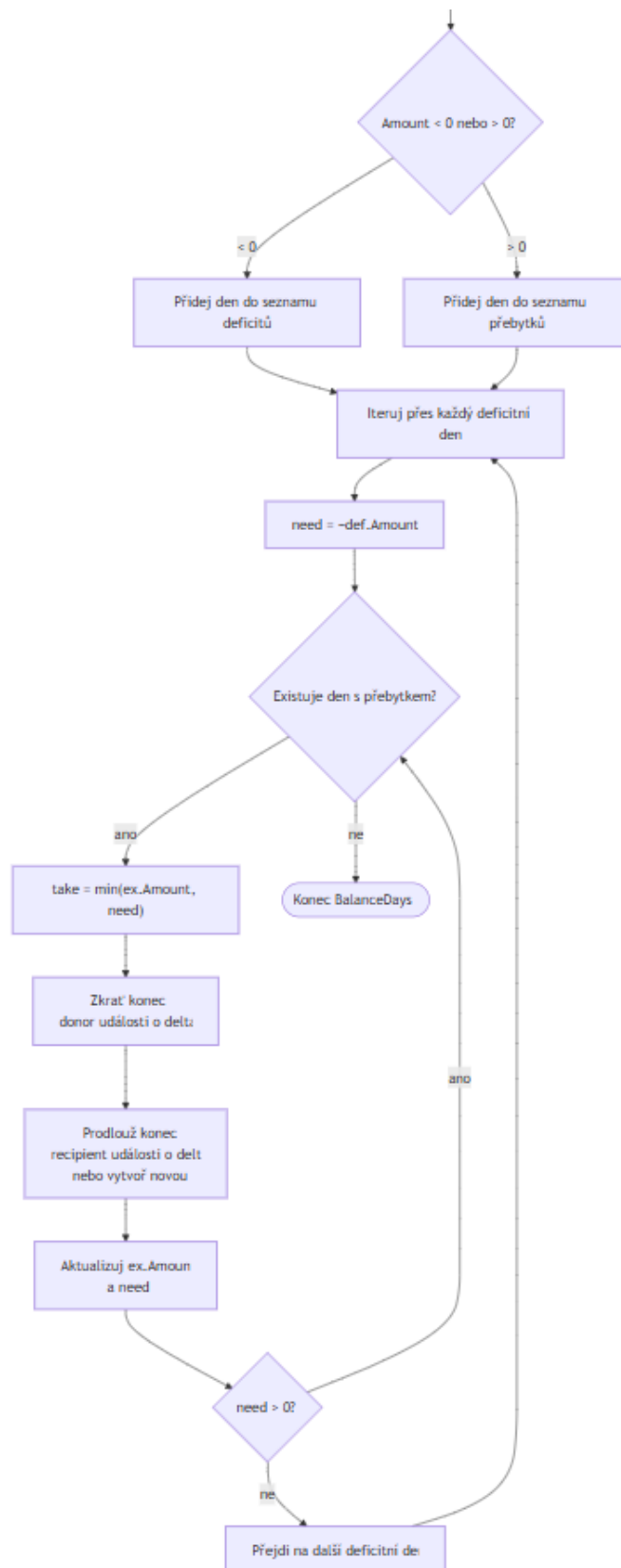
Jedním z klíčových mechanismů implementovaných v aplikaci je algoritmus BalanceDays, jehož cílem je zajistit rovnováhu mezi očekávanými a skutečně odpracovanými hodinami v rámci celého měsíce. Postup začíná načtením všech událostí z databáze pro zvolený měsíc a jejich rozdělením podle jednotlivých pracovních dnů. Víkendy a státní svátky jsou z procesu automaticky vyřazeny, protože se nepovažují za pracovní dny. Pro každý den je vytvořena datová struktura, která uchovává související události a slouží jako základ pro další výpočty.

Ve druhé fázi algoritmus určuje dvě zásadní hodnoty. První z nich je očekávaný počet hodin, jenž vychází z globálních nastavení, nebo v případě, že uživatel zadal individuální parametry, právě z těchto dat. Očekávaná doba se získá z rozdílu mezi časem příchodu a odchodu, přičemž je zohledněna délka obědové přestávky. Druhou hodnotou je skutečně odpracovaný čas, který je určen na základě událostí typu „Work“ nebo „BusinessTrip“. Tyto časové bloky jsou nejprve sloučeny, aby se eliminovaly možné překryvy, a výsledkem je přesná délka pracovního dne.

Následuje krok, v němž se porovnává skutečná a očekávaná hodnota. Pokud se zjistí, že je odpracovaný čas vyšší než norma, vzniká přebytek, pokud je naopak nižší, eviduje se deficit. Algoritmus se následně pokouší o vyrovnání rozdílů tím, že přebytečné hodiny z vybraných dnů postupně přenáší do těch deficitních. Tento proces probíhá tak, že z „dárcovských“ dnů se ubírá část pracovních bloků a tyto hodiny se naopak přidávají dnům, kde hodiny chybí. Pokud by přidělené hodiny kolidovaly s obědovou přestávkou, algoritmus automaticky vloží novou pauzu, aby byl zachován správný rytmus dne. Po dokončení redistribuce se nová nastavení uloží do databáze a aktualizují se také samotné události.

Výsledkem tohoto algoritmu je skutečnost, že v souhrnu za celý měsíc se odpracovaný čas přiblíží co nejvíce očekávané normě a případné rozdíly mezi jednotlivými dny se vyrovnají.





Obrázek 11 : Blokový diagram algoritmu

2.7.2 Popis algoritmu ze služby pro výpočet pracovní doby

Další skupinu tvoří algoritmy určené pro výpočet pracovní doby. Jejich úlohou je určit, kolik hodin má uživatel skutečně odpracováno v jednotlivých časových horizontech – od denních souhrnů až po měsíční bilance.

Základem je metoda, která stanovuje očekávaný počet hodin pro každý den. Nejprve se ověřuje, zda se nejedná o víkend nebo svátek, protože v takovém případě je výsledek nulový. V pracovních dnech se pak využívají hodnoty uložené v globálních nastaveních nebo v individuálních preferencích uživatele. Na jejich základě se vypočítá pracovní interval a od něj se odečte doba přestávky. Výsledkem je norma, kterou by měl uživatel daný den splnit.

Skutečně odpracovaná doba se určuje na základě jednotlivých událostí. Pokud se jedná o záznam typu „Work“ nebo „BusinessTrip“, do výpočtu se zahrne délka mezi časem začátku a konce. Naopak přestávky a jiné pomocné bloky se ignorují. Překrývající se časové úseky se sjednocují, aby nedošlo k duplicitnímu započítání. Takto získaný součet tvoří reálnou pracovní dobu.

Na základě denních výsledků lze sestavit týdenní bilanci. Algoritmus zde vychází z pondělí jako počátečního dne a zahrnuje pět pracovních dnů. Pro každý z nich vypočítá jak normu, tak skutečný odpracovaný čas. Součástí výsledku je i vyčíslení přesčasů a deficitů, což poskytuje uživateli komplexní přehled o tom, jak se mu daří plnit očekávaný fond hodin.

Podobným způsobem je konstruován i měsíční algoritmus. Ten nejen sumarizuje všechny dny, ale rovněž určuje, kolik hodin bylo odpracováno nad rámec a kolik jich naopak chybí. Výsledkem je model, který umožňuje přebytky a deficity mezi jednotlivými dny přerozdělovat, což v kombinaci s algoritmem BalanceDays zajišťuje konzistenci celkového součtu.

ZÁVĚR

Cílem této bakalářské práce bylo navrhnout, implementovat a ověřit multiplatformní desktopovou aplikaci pro digitalizaci evidence pracovní doby vyučujících, která automatizuje zpracování vstupních dat (rozvrh ze STAG, přestávky, školní prázdniny a státní svátky) a generuje validní PDF dokument připravený k odevzdání. V teoretické části práce jsme analyzovali klíčové technologie a návrhové vzory využívané při vývoji aplikace. Popisovali jsme programovací jazyk C#, relační databázi SQLite a deklarativní značkovací jazyk .axaml v Avalonia UI. Dále jsme představili vývojové prostředí Microsoft Visual Studio a SQLiteStudio a přiblížili architekturu desktopové aplikace založenou na MVVM. Podrobně jsme rozebrali využívané NuGet balíčky (Avalonia, ReactiveUI, EF Core, QuestPDF a další), které zajišťují robustnost, přenositelnost a reaktivitu řešení.

V praktické části práce jsme detailně popsali strukturu SQLite databáze se třemi tabulkami (Events, GlobalSettings, UserSettings), uspořádání projektové složkové struktury dle principu Separation of Concerns a implementaci prezentační vrstvy v Avalonia UI. Klíčovou součástí byla také realizace algoritmu pro vyvážení odpracovaných hodin, který se spouští při generování náhledu PDF a automaticky upravuje časy událostí tak, aby odpovídaly globálním či uživatelským nastavením.

Výsledná aplikace poskytuje uživatelsky přívětivé rozhraní pro rychlé zadávání a úpravy dat, vizualizaci denního, týdenního i měsíčního kalendáře a okamžitý náhled výsledného PDF dokumentu. Díky použití Avalonia UI běží plynule jak na Windows, tak na Linuxu, a díky Entity Framework Core se SQLite nabízí spolehlivé a přenosné datové úložiště.

Navržené řešení významně zkracuje čas potřebný k přípravě měsíčních a týdenních vyúčtování pracovní doby a minimalizuje riziko chyb při manuálním přepisování dat. Aplikace tak přispívá ke zvýšení administrativní efektivity a transparentnosti správy pracovního fondu na akademických pracovištích. V budoucnu je možné řešení rozšířit o jednotnou autentizaci (SSO), auditní logování či export do dalších formátů, což dále posílí jeho použitelnost a škálovatelnost.

POUŽITÁ LITERATURA

1. MICROSOFT, 2024. Dokumentace k jazyku C#. In: learn.microsoft.com [online]. Microsoft, 2024 [cit. 2025-05-04]. Dostupné z: <https://learn.microsoft.com/cs-cz/dotnet/csharp/tour-of-csharp/>
2. SQLITE, 2025. Oficiální dokumentace SQLite. In: sqlite.org [online]. SQLite, 2025 [cit. 2025-05-04]. Dostupné z: <https://www.sqlite.org/lang.html>
3. AVALONIA UI, 2025. Dokumentace Avalonia UI. In: docs.avaloniaui.net [online]. Avalonia, 2025 [cit. 2025-05-04]. Dostupné z: <https://docs.avaloniaui.net/>
4. QUESTPDF, 2025. Oficiální dokumentace QuestPDF. In: questpdf.com [online]. QuestPDF, 2025 [cit. 2025-05-04]. Dostupné z: <https://www.questpdf.com/>
5. MICROSOFT, 2024. Entity Framework Core v .NET – práce s databázemi SQLite. In: learn.microsoft.com [online]. Microsoft, 2024 [cit. 2025-05-04]. Dostupné z: <https://learn.microsoft.com/cs-cz/ef/core/providers/sqlite/>
6. ARTIFEX SOFTWARE, 2025. Ghostscript – oficiální dokumentace. In: ghostscript.com [online]. Artifex Software, 2025 [cit. 2025-05-04]. Dostupné z: <https://www.ghostscript.com/>
7. HABJAN, John a kol., 2025. Ghostscript.NET – .NET wrapper pro Ghostscript. In: github.com/jhabjan/Ghostscript.NET [online]. Ghostscript.NET, 2025 [cit. 2025-05-04]. Dostupné z: <https://github.com/jhabjan/Ghostscript.NET>
8. AVALONIA XAML BEHAVIORS, 2024. GitHub – Avalonia.XamlBehaviors. In: github.com/AvaloniaXAMLBehaviors [online]. Avalonia Team, 2024 [cit. 2025-05-04]. Dostupné z: <https://github.com/AvaloniaXAMLBehaviors>
9. AVALONIA COMMUNITY, 2024. Práce s chováním (Behaviors) v Avalonia UI. In: docs.avaloniaui.net [online]. Avalonia Community, 2024 [cit. 2025-05-04]. Dostupné z: <https://docs.avaloniaui.net/docs/controls/behaviors>
10. REACTIVEUI COMMUNITY, 2024. ReactiveUI – dokumentace a postupy. In: reactiveui.net [online]. ReactiveUI Community, 2024 [cit. 2025-05-04]. Dostupné z: <https://www.reactiveui.net/docs/getting-started/>
11. MICROSOFT, 2023. Data binding a chování v Avalonia a WPF (Behaviors). In: learn.microsoft.com [online]. Microsoft, 2023 [cit. 2025-05-04]. Dostupné z: <https://learn.microsoft.com/cs-cz/dotnet/desktop/wpf/advanced/behaviors-overview>

SEZNAM PŘÍLOH

Příloha A: Zdrojový kód aplikace TeacherScheduleApp

PŘÍLOHA A: Zdrojový kód aplikace TeacherScheduleApp

Tato příloha obsahuje zip archiv s projektem z Visual Studio 2022 jménem TeacherScheduleApp